

UNIT 5: EXAMINING SELF- DEFENDING MALWARE

Dhanashree Kulkarni

Index



- ❑ How Malware Detects Debuggers and Protects Embedded Data
- ❑ Unpacking Malicious Software that Employs Process Hollowing
- ❑ Bypassing the Attempts by Malware to Detect and Evade the Analysis Toolkit.
- ❑ Handling Code Misdirection Techniques, Including SEH and TLS Callbacks.
- ❑ Unpacking Malicious Executable by Anticipating the Packer's Actions.

Overview

- Newer ways of not getting detected are being employed by attackers all around the globe.
- Using **evasion techniques**, these attackers can be **invisible to threat analysis professionals** as well.
- By **hiding indicators of malicious activities**, such attacks **continue for a longer period**. One of the common methods employed **to stop attacks getting identified** is **anti-debugging, which ensures that a program is not running under a debugger**.
- Anti-debugging is the implementation of one or more techniques within **computer code that delays attempts at reverse engineering or debugging a target process**
- Once a debugger is detected, the problem doesn't stop there. The minute the malware gets to know that it has been detected by a debugger, it changes its behavior, adjusts the code execution path or simply modifies itself to evoke a crash. Once the system crashes, it is back to square one for the cyber security analysts and experts.

Anti-Debugging

- When **developers** are working on their codes, they use some **best practices to protect their code and ultimately the application that they are writing the code for.**
- A **debugger** helps them **test** their **code** in **controlled environments** against **known attacks**, change **certain variables**, **upgrade configurations** etc. **Bugs, errors** and **security loopholes** can be **tested** and **fixed**, so that **data** is ultimately **protected**.
- However, even though they are useful for developers, these **debuggers** can themselves open up the **playground** for **attackers** to **observe** the **application** being **tested** under **specific conditions**, thereby **helping** them **better their attack techniques**.
- Anti-debugging can help change the behavior of the apps to protect themselves and, in most cases, slow down the reverse engineering process. Some common methods used in anti-debugging techniques are:
 - ▣ Analysis of data being exchanged between applications using a packet sniffer
 - ▣ Disassembly of software binary code so it gets added in the assembly language
 - ▣ Recreating source code by decompiling binary or byte-code

Best Techniques of Anti-Debugging

- Looking for modifications in the code
 - ▣ Debugging usually relies on modifying the original code by changing instructions from code strings. Thus, the app can be programmed to identify debuggers in such cases and have defensive techniques ready. (<https://www.apriorit.com/dev-blog/367-anti-reverse-engineering-protection-techniques-to-use-before-releasing-software>)
- Adding exceptions in code
 - ▣ When certain exceptions are not executed, it most likely would mean that a debugger is present. A good cybersecurity professional should be able to manage this
- Self-debugging
 - ▣ When an application is running, it is possible to create a new process that can get attached to the debugger of the original parent process. Now if another debugger is running, an alert will get generated. This might be a more strenuous process since it is to be started from scratch, but it is worth the effort.

- Using system API calls

- ▣ APIs are commonly provided with every application. Using these would be the easiest to know whether a debugger is running in the background.

- Exploiting bugs in popular debuggers

- ▣ Debuggers are not always perfect, and this technique uses that point to its advantage. Every application has some flaw, and so is the case with every debugger. By introducing data values or specific instructions that can cause a crash or make the system behave in a certain manner, a debugger can be revealed.

- Using hardware breakpoint detection

- ▣ Breakpoints are meant to test for certain signals, and similarly, when encountered by a developer, they can help detect a debugger too

□ Relying on timing-based detection

- ▣ Common sense tells us that an application is usually expected to execute in a set time interval. Developers are usually aware of the different steps and procedures which make up an application and its features. So a lag in execution of certain steps can give a clue towards a debugger.
- ▣ Of course, when multiple applications are running or interacting with each other, with different time intervals for action between them, this could get a bit tricky.
- ▣ The best way out would be to keenly observe the time intervals between processes and interactions to see any lags/delays which could be attributed to a debugger

- Keeping an eye on certain libraries

- ▣ Debuggers usually come with their specific set of libraries. A simple look at the list of libraries attached to an application can reveal the presence or absence of a debugger

- Mixing and matching technique

- ▣ A good mix of different techniques is what will ultimately come in handy when you want to make the lives of attackers a tad bit difficult. Reverse engineering a code will become all the more difficult when you confuse the attacker.

Some other functions which help with anti-debugging are:

- ❑ **IsDebuggerPresent:** One of the simplest methods, this function checks if a calling program is being debugged by a user-mode debugger. Calling this function will help stop the debugger.
- ❑ **PEB (Process Environment Block):** This is used in the Windows OS.
- ❑ **TLS Callback:** The main function of an application is usually the place which is targeted by attackers and reverse engineering experts. Hence, performing a debugger check there may not be that fruitful. Also, the checks can be erased. The TLS Callback is thus a better place for checking for debuggers.
- ❑ **NtGlobalFlag:** When it comes to system tracking, debugging and control, the global variable NtGlobalFlag is useful.

Unpacking Malicious Software that Employs Process Hollowing

- Process hollowing is a security exploit in **which an attacker removes code in an executable file and replaces it with malicious code**. The process hollowing attack is used by hackers to **cause an otherwise legitimate process to execute malicious code**.
- Adversaries may inject malicious code into suspended and hollowed processes in order to evade process-based defenses.
- Process hollowing is a method of executing arbitrary(random) code in the address space of a separate live process.
- Process hollowing is commonly performed by creating a process in a **suspended state** then unmapping/hollowing its memory, which can then be replaced with malicious code.
- A victim process can be created with native Windows API calls such as CreateProcess, which includes a flag to suspend the processes primary thread.
- At this point the process can be unmapped using APIs calls such as ZwUnmapViewOfSection or NtUnmapViewOfSection before being written to, realigned to the injected code, and resumed via VirtualAllocEx, WriteProcessMemory, SetThreadContext, then ResumeThread respectively.

Procedure Examples

ID	Name	Description
S0331	Agent Tesla	Agent Tesla has used process hollowing to create and manipulate processes through sections of unmapped memory by reallocating that space with its malicious code.
S0373	Astaroth	Astaroth can create a new process in a suspended state from a targeted legitimate process in order to unmap its memory and replace it with malicious code.
S0344	Azorult	Azorult can decrypt the payload into memory, create a new suspended process of itself, then inject a decrypted payload to the new process and resume new process execution.
S0128	BADNEWS	BADNEWS has a command to download an .exe and use process hollowing to inject it into a new process.
S0234	Bandook	Bandook has been launched by starting iexplore.exe and replacing it with Bandook 's payload.
S0534	Bazar	Bazar can inject into a target process including Svchost, Explorer, and cmd using process hollowing.
S0127	BBSRAT	BBSRAT has been seen loaded into msixexec.exe through process hollowing to hide its execution.
S0660	Clambling	Clambling can execute binaries through process hollowing.
S0154	Cobalt Strike	Cobalt Strike can use process hollowing for execution.

Techniques Addressed by Mitigation

Domain	ID	Name	Use
Enterprise	<u>T1059</u>	<u>Command and Scripting Interpreter</u>	On Windows 10, enable Attack Surface Reduction (ASR) rules to prevent <u>Visual Basic</u> and <u>JavaScript</u> scripts from executing potentially malicious downloaded content
		<u>.005</u>	On Windows 10, enable Attack Surface Reduction (ASR) rules to prevent <u>Visual Basic</u> scripts from executing potentially malicious downloaded content
		<u>.007</u>	On Windows 10, enable Attack Surface Reduction (ASR) rules to prevent <u>JavaScript</u> scripts from executing potentially malicious downloaded content
Enterprise	<u>T1543</u>	<u>Create or Modify System Process</u>	On Windows 10, enable Attack Surface Reduction (ASR) rules to prevent an application from writing a signed vulnerable driver to the system. ^[2] On Windows 10 and 11, enable Microsoft Vulnerable Driver Blocklist to assist in hardening against third party-developed drivers.
		<u>.003</u>	On Windows 10, enable Attack Surface Reduction (ASR) rules to prevent an application from writing a signed vulnerable driver to the system. ^[2] On Windows 10 and 11, enable Microsoft Vulnerable Driver Blocklist to assist in hardening against third party-developed service drivers.

Attack surface reduction rules

- Organization's attack surface includes **all the places where an attacker could compromise your organization's devices or networks**. Reducing your attack surface means protecting your organization's devices and network, which leaves attackers with fewer ways to perform attacks. Configuring attack surface reduction rules in **Microsoft Defender for Endpoint** can help!
- Attack surface reduction rules target certain software behaviors, such as:
 - ▣ **Launching executable** files and scripts that attempt to **download or run files**
 - ▣ **Running** obfuscated or otherwise **suspicious scripts**
 - ▣ Performing **behaviors that apps don't usually initiate during normal day-to-day work**

Bypassing the Attempts by Malware to Detect and Evade the Analysis Toolkit.

- Sandboxing technology is widely used for malware detection and prevention, so **hackers search for ways to teach their malware to stay inactive in the sandbox.** In this way, sandbox-evading malware can bypass protections and execute malicious code without being detected by modern cybersecurity solutions.
- Sandboxing is an automated technology for malware detection that's widely used by traditional antivirus programs and other security applications. **By placing a potentially dangerous program into a controlled virtualized environment where it can't cause any harm, security software can analyze the behavior of the malware and develop protection against it.**
- Though sandboxing technology has been considered effective, cybercriminals are now applying new techniques that let malware evade sandbox analysis.
- According to some estimates, nearly 98% of modern malware uses at least one evasive technique. In addition, more than 32% of malware is armed with six or more methods for avoiding detection in antivirus sandboxes.

What is sandbox-evading malware?

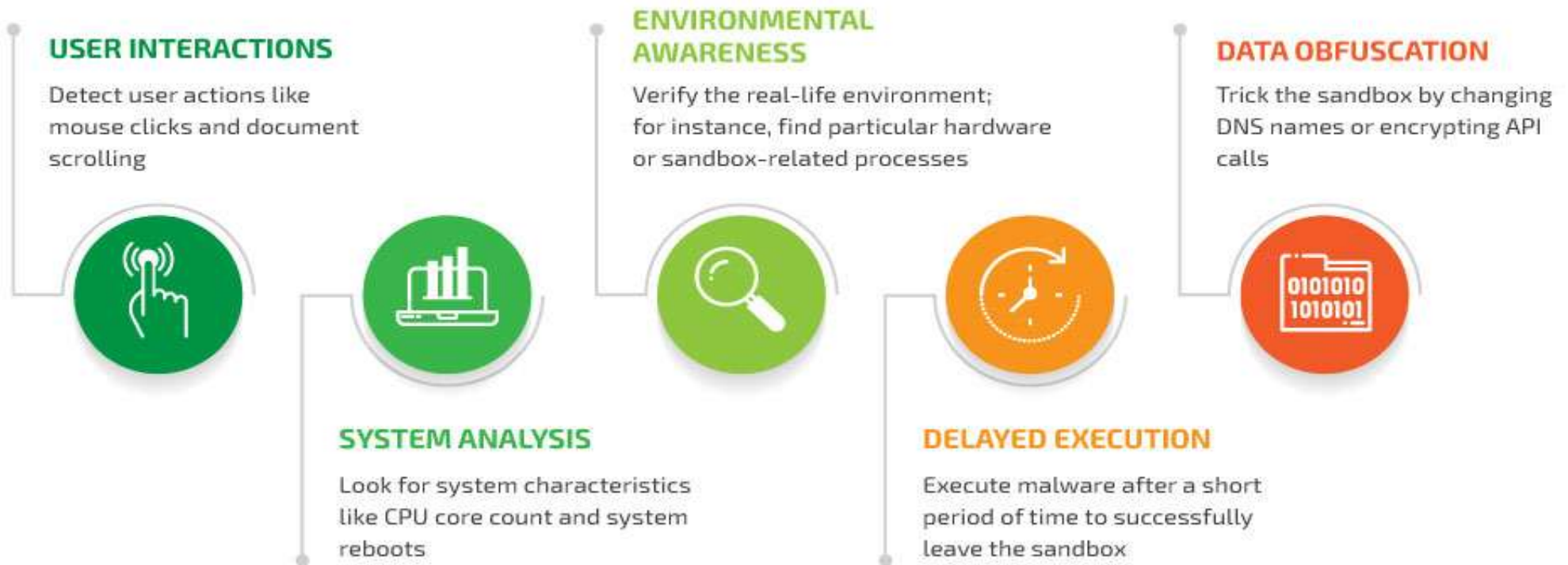
- Sandbox-evading malware is a new type of malware that can recognize if it's inside a sandbox or virtual machine environment. These malware infections don't execute their malicious code until they're outside of the controlled environment.
- The first malware that bypassed sandbox protection appeared in the 1980s. This malware used an evasion technique that allowed the virus to encrypt its own code so security specialists were unable to read it. Since then, cybercriminals have developed more than 500 evasion techniques and several contemporary malware families based on them.
- Nowadays, encrypted malware can hide complex threats aimed to attack a victim's system over an extended period of time. Moreover, there are entirely new types of malware with sandbox evasion techniques that can bypass even those protection technologies that use machine learning algorithms.
 - **To avoid detection, malware uses special sandbox evasion techniques that are mainly based on either detecting user or system interactions or obtaining environmental awareness.**

Examples of sandbox-evading malware

- ❑ **Locky ransomware**, released in 2016, is a classic example of a sandbox-evading virus. Locky was spread through JavaScript code that was infected with encrypted DLL files. This malware requires the use of run32dll.exe to execute the DLL. However, run32dll.exe isn't available in the sandbox environment, and their absence allows the malware to remain undetected. Another malware from the Locky family, KeRanger, begins to act only three days after it's downloaded to the system.
- ❑ In the middle of 2018, a new version of the **RogueRobin** trojan was detected in the Middle East. This government-oriented malware was spread via email in an attached RAR archive. Once started, the malware used a virtual machine image to verify the BIOS version, CPU cores, and physical memory of the system on which it was running and wouldn't run in a sandbox.
- ❑ In 2019, hackers used the **HAWKBALL** backdoor to attack the government sector in Central Asia. This malware exploited Microsoft Office vulnerabilities to deliver payloads and collect system information. It was able to execute native Microsoft commands, survey hosts, and even verify if the process it used was being debugged.
- ❑ In March 2019, a new sample of **macOS malware** using improved sandbox evading techniques was detected. In contrast to its predecessors, OSX_OCEANLOTUS.D had a Mach-O signature with a UPX string that allowed it to be unnoticeable during static analysis in a virtual environment. Besides, this backdoor executable checked if it was connected to a debugger and gathered data about the host machine.

Most common sandbox evasion techniques

Common sandbox evasion techniques:



Detecting user interactions

- Users interact with computer systems in different ways but there are **no human-like interactions in the sandbox environment**. Thus, **hackers can teach malware to wait for a specific user action and exhibit malicious behavior only afterward**. Here are some examples of what user actions can activate sandbox-evading malware:
- **Scrolling a document.** Modern malware can be programmed to execute only after scrolling to a particular place in a document. For instance, one piece of malware activated only after a user scrolled an RTF document to the second page. To detect this, the malware contained paragraph codes used in Microsoft Word documents. While RTF files also include paragraph marks (./par), the malware code contained a series of paragraphs that needed to be scrolled before the exploit code executed. The sandbox environment doesn't contain any scroll movements, allowing the malware to stay dormant.
- **Moving and clicking the mouse.** Some malware is programmed to check the speed of mouse movements and clicks and remain inactive if the speed is suspiciously fast. Trojan.APT.BaneChan, for instance, activates only after a certain number of mouse clicks made by a user.

Detecting system characteristics

- ❑ The **core count technique** allows malware to **find discrepancies between virtual and physical systems**, such as the **number of CPU cores**. This is why many sandbox vendors hide their actual configurations so hackers are unable to detect sandbox specifications.
- ❑ **Digital system signature**. Some malware is designed to find the digital signature of a system, which contains information about the computer's configuration.
- ❑ **Installed programs**. This technique allows malware to check the availability of antivirus programs by looking for active processes in the operating system. The Client Maximus malware is an example of sandbox-evading malware that applies this technique using a stealthy driver.
- ❑ **Operating system reboots**. Malware may be scheduled to activate only when the system reboots. This technique is used because some sandboxes can't survive reboots. Though the virtual environment may try to emulate a reboot by logging the user in and out, malware can detect this, as not all reboot triggers are executed.

Environmental awareness

Environmental awareness

Sandbox-related characteristics

- Sandbox usernames
- Hypervisor calls
- Sandbox processes

Real-life environment characteristics

- Devices installed
- Breakpoint registers
- Dynamic link library

- ❑ To check its environment, malware can be programmed to detect devices installed on the infected system or look for indicators that belong only to a **virtual environment, such as hypervisor calls, certain file names, and processes typical of a sandbox.**
- ❑ Besides, such malware can detect the sandbox when it's named as "sample" or "malware." In addition, a virus can detect that it's in a virtual environment **when it finds such processes as vmusrvc.exe, boxservice.exe, or vmttoolsd.exe.**
- ❑ For example, the BlackSquid malware designed to mine Monero cryptocurrency used a range of methods to become aware of its environment. Particularly, this malware checked for **common sandbox usernames and breakpoint registers for hardware breakpoints.** It also verified **disk drive models, device drivers, and dynamic link libraries.**

Other evasion techniques

- Timing-based techniques
- In some cases, malware evades the sandbox using timing-based techniques. Sandboxes usually analyze malware only for a limited period of time, and timing-based techniques gladly abuse this feature.
- These are techniques where malware **delays its malicious activity** to avoid detection.
- Three common types of time-based sandbox-evading techniques:
 - ▣ **Extended sleep.** When malware uses calls for extended sleep, it can successfully leave the sandbox before execution.
 - ▣ **Logic bomb.** In some cases, malware can be programmed to execute on a particular date and at a particular time.
 - ▣ **Stalling code.** Malware can contain malicious code that executes useless CPU cycles to delay the actual code until the sandbox has finished testing.

- Obfuscating internal data
- There are some sandbox evasion techniques that allow malware to change or encrypt its code and communications so that the sandbox can't analyze it.
 - ▣ **Fast flux.** This technique is based on **changing DNS names and IP addresses and is widely used by botnets that want to hide phishing and malware delivery addresses**. It allows malware to bypass the blacklist of malware websites that security solutions create. For instance, the Necurs botnet changes website addresses by using a domain generation algorithm that creates new domain names every four days.
 - ▣ **Data encryption.** Some malware, like Trojan Dridex, can also encrypt API calls so that traditional malware sandboxes can't read them. The Andromeda botnet used several keys to encrypt its communication with the server.

How to detect sandbox-evading malware

- **Dynamically change sleep duration.** While a sandbox usually analyzes malware for seconds, a prolonged analysis significantly increases the chances for detecting malware with increased sleep duration. However, this approach may not be effective, as it requires more time. Instead, you can make the sandbox dynamically change its time settings to deceive malware and stimulate its execution.
- **Simulate human interactions.** The sandbox environment doesn't simulate interactions by default, but you may add some user-like interactions to better analyze malware. However, keep in mind that modern malware may be clever enough to detect fake mouse clicks or movements.
- **Add real environmental and hardware artifacts.** Retrieving hardware information in your sandbox will help you detect malware that checks for hard disk size, recent files, CPU numbers, operating system version, memory volume, and other system and hardware characteristics.
- **Perform static in addition to dynamic analysis.** Sandboxing technology is a form of dynamic malware analysis, as it examines malware behavior in a safe environment. While sandbox-evading malware doesn't perform any actions, you can subject it to full static code analysis. Static analysis will check the file for evasion techniques or encrypted pieces of code.

- **Use fingerprint analysis.** Fingerprinting technology allows you to analyze a malware file and find indicators of malicious code. Fingerprinting can also be used for detecting evasion characteristics of malware.
- **Use behavior-based analysis.** Behavior-based analysis offers features designed to detect and combat evasion techniques. During this analysis, the sandbox interacts with the malware itself to find possible execution paths. Moreover, it emulates process interactions to look like a host computer. Once a sandbox evasion technique is detected, the sandbox counteracts its malicious code.
- **Customize your sandboxing.** By adding other innovative features for malware detection to the sandbox, you can significantly improve its effectiveness at detecting malware. For instance, you can use a multi-sandbox array of diverse environments and iterative analysis. It's also effective to check malware communications beyond the machine's system API. You can also add a feature to your sandbox that searches for and verifies traces of malicious code at runtime.

- **Add kernel analysis.** While most sandbox solutions operate in user mode, some types of malware are designed to inject malicious code into the kernel space (rootkits or drivers) and thus escape sandboxing. For instance, the early versions of Turla malware loaded and exploited a vulnerable VirtualBox driver and disabled checks for signed driver loading. Thus, by adding kernel analysis to your solution, you can prevent malware from moving into the kernel.
- **Implement machine learning.** Malware analysis based on machine learning algorithms can effectively detect sandbox evasion techniques in malware code before it executes. Machine learning algorithms can analyze every act of malware inactivity or sleep as a signal of an evasion technique. Moreover, it can collect millions of other signals that collectively can detect malicious code.
- **Consider content disarm and reconstruction (CDR)** as an extra security layer. CDR is often considered the opposite of sandboxing, but it may serve as an add-on to other security solutions. This technology removes all active content from a file and provides a user with a sanitized document. It allows you to instantly prevent malware hidden in documents, yet there's a risk of getting corrupt files containing scripts (such as Office macros written in JavaScript) even though they're not malicious